01 LC 136 | Single Number (Easy)

Problem: Given a non-empty array of integers nums, every element appears twice except for one. Find that single one. You must implement a solution with a linear runtime complexity and use only constant extra space.

Constraints: 1 <= nums.length <= 3 * 10^4; -3 * 10^4 <= nums[i] <= 3 * 10^4; Each element in the array appears twice except for one element which appears only once.

```
class Solution {
public:
  int singleNumber(vector<int>& nums) {
    int ans = 0;
    for (int v : nums) {
      ans ^= v;
    }
    return ans;
  }
};
```

14 LC 260 | Single Number III (Medium)

Problem: Given an integer array nums, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in any order. You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Constraints: 2 <= nums.length <= 3 * 10^4; -2^31 <= nums[i] <= 2^31 - 1; Each integer in nums will appear twice, only two integers will appear once.

```
class Solution {
public:
  vector<int> singleNumber(vector<int>& nums) {
    long long xs = 0;
    for (int& x : nums) {
      xs ^= x;
    }
    int lb = xs & -xs;
    int a = 0;
    for (int& x : nums) {
      if (x & lb) {
        a ^= x;
      }
    }
    int b = xs ^ a;
    return {a, b};
  }
};
```

48 LC 149 | Max Points on a Line (Hard)

Problem: Given an array of points where points[i] = [x_i, y_i] represents a point on the X-Y plane, return the maximum number of points that lie on the same straight line.

Constraints: 1 <= points.length <= 300; points[i].length == 2; -10^4 <= x_i, y_i <= 10^4; All the points are unique.

```
class Solution {
public:
  int maxPoints(vector<vector<int>>& points) {
    int best = 1, n = points.size();
    for (int i = 0; i < n; ++i) {
      unordered_map<string, int> count;
      for (int j = i + 1; j < n; ++j) {
        int dx = points[j][0] - points[i][0], dy =
          points[j][1] - points[i][1];
        int divisor = std::gcd(dx, dy);
        dx /= divisor; dy /= divisor;
        if (dx < 0 || (dx == 0 && dy < 0)) { dx =
          -dx; dy = -dy; }
        string key = to_string(dx) + ":" +
          to_string(dy);
        best = max(best, 1 + ++count[key]);
      }
    }
    return best;
  }
};
```

02 LC 191 | Number of 1 Bits (Easy)

Problem: Given a positive integer n, write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Constraints: 1 <= n <= 2^31 - 1

```
class Solution {
public:
  int hammingWeight(uint32_t n) { return
    std::popcount(n); }
};
```

27 LC 415 | Add Strings (Easy)

Problem: Given two non-negative integers, num1 and num2 represented as string, return the sum of num1 and num2 as a string. You must solve the problem without using any built-in library for handling large integers (such as BigInteger). You must also not convert the inputs to integers directly.

Constraints: 1 <= num1.length, num2.length <= 10^4; num1 and num2 consist of only digits.; num1 and num2 don't have any leading zeros except for the zero itself.

```
class Solution {
public:
  string addStrings(string num1, string num2) {
    int i = num1.size() - 1, j = num2.size() - 1;
    string ans;
    for (int c = 0; i >= 0 || j >= 0 || c; --i,
      --j) {
      int a = i < 0 ? 0 : num1[i] - '0';
      int b = j < 0 ? 0 : num2[j] - '0';
      c += a + b;
      ans += to_string(c % 10);
      c /= 10;
    }
    ranges::reverse(ans);
    return ans;
  }
  string subStrings(string num1, string num2) {
    int m = num1.size(), n = num2.size();
    bool neg = m < n || (m == n && num1 < num2);
    if (neg) {
      swap(num1, num2);
    }
    int i = num1.size() - 1, j = num2.size() - 1;
    string ans;
    for (int c = 0; i >= 0; --i, --j) {
      c = (num1[i] - '0') - c - (j < 0 ? 0 :
        num2[j] - '0');
      ans += to_string((c + 10) % 10);
      c = c < 0 ? 1 : 0;
    }
    while (ans.size() > 1 && ans.back() == '0') {
      ans.pop_back();
    }
    if (neg) {
      ans.push_back('-');
    }
    ranges::reverse(ans);
    return ans;
  }
};
```

03 LC 338 | Counting Bits (Easy)
Problem: Given an integer n, return an array ans of length n + 1 such that for each i (0 <= i <= n), ans[i] is the number of 1's in the binary representation of i. Do not solve it with built-in functions (i.e., like __builtin_popcount in C++).
Constraints: 0 <= n <= 10^5

```
class Solution {
public:
  vector<int> countBits(int n) {
    vector<int> ans(n + 1);
    for (int i = 1; i <= n; ++i) {
      ans[i] = ans[i & (i - 1)] + 1;
    }
    return ans;
  }
};
```

33 LC 171 | Excel Sheet Column Number (Easy)
Problem: Given a string columnTitle that represents the column title as appears in an Excel sheet, return its corresponding column number. For example: A -> 1 B -> 2 C -> 3 ... Z -> 26 AA -> 27 AB -> 28 ...
Constraints: 1 <= columnTitle.length <= 7; columnTitle consists only of uppercase English letters.; columnTitle is in the range ["A", "FXSHRXW"].

```
class Solution {
public:
  int titleToNumber(string columnTitle) {
    int ans = 0;
    for (char& c : columnTitle) {
      ans = ans * 26 + (c - 'A' + 1);
    }
    return ans;
  }
};
```

41 LC 1318 | Minimum Flips to Make a OR b Equal to c (Medium)
Problem: Given 3 positives numbers a, b and c. Return the minimum flips required in some bits of a and b to make (a OR b == c). (bitwise OR operation). Flip operation consists of change any single bit 1 to 0 or change the bit 0 to 1 in their binary representation.
Constraints: 1 <= a <= 10^9; 1 <= b <= 10^9; 1 <= c <= 10^9

```
class Solution {
public:
  int minFlips(int a, int b, int c) {
    int ans = 0;
    for (int i = 0; i < 32; ++i) {
      int x = a >> i & 1, y = b >> i & 1, z = c >>
        i & 1;
      ans += z == 0 ? x + y : (x == 0 && y == 0 ?
        1 : 0);
    }
    return ans;
  }
};
```

49 LC 223 | Rectangle Area (Medium)
Problem: Given the coordinates of two rectilinear rectangles in a 2D plane, return the total area covered by the two rectangles. The first rectangle is defined by its bottom-left corner (ax1, ay1) and its top-right corner (ax2, ay2). The second rectangle is defined by its bottom-left corner (bx1, by1) and its top-right corner (bx2, by2).
Constraints: -10^4 <= ax1 <= ax2 <= 10^4; -10^4 <= ay1 <= ay2 <= 10^4; -10^4 <= bx1 <= bx2 <= 10^4; -10^4 <= by1 <= by2 <= 10^4

```
class Solution {
public:
  int computeArea(int ax1, int ay1, int ax2, int
    ay2, int bx1, int by1, int bx2, int by2) {
    int a = (ax2 - ax1) * (ay2 - ay1);
    int b = (bx2 - bx1) * (by2 - by1);
    int width = min(ax2, bx2) - max(ax1, bx1);
    int height = min(ay2, by2) - max(ay1, by1);
    return a + b - max(height, 0) * max(width, 0);
  }
};
```

04 LC 190 | Reverse Bits (Easy)
Problem: Reverse bits of a given 32 bits signed integer.
Constraints: 0 <= n <= 2^31 - 2; n is even.

```
class Solution {
public:
  uint32_t reverseBits(uint32_t n) {
    uint32_t ans = 0;
    for (int i = 0; i < 32 && n; ++i) {
      ans |= (n & 1) << (31 - i);
      n >>= 1;
    }
    return ans;
  }
};
```

11 LC 461 | Hamming Distance (Easy)
Problem: The Hamming distance between two integers is the number of positions at which the corresponding bits are different. Given two integers x and y, return the Hamming distance between them.
Constraints: 0 <= x, y <= 2^31 - 1

```
class Solution {
public:
  int hammingDistance(int x, int y) { return
    popcount(static_cast<unsigned>(x ^ y)); }
};
```

42 LC 2220 | Minimum Bit Flips to Convert Number (Easy)
Problem: One operation flips any one bit. Return the minimum bit flips needed to convert start to goal.
Constraints: 0 <= start, goal <= 10^9

```
class Solution {
public:
  int minBitFlips(int start, int goal) {
    return popcount(static_cast<unsigned
      int>(start ^ goal));
  }
};
```

44 LC 1442 | Count Triplets That Can Form Two Arrays of Equal XOR (Medium)
Problem: Count triples 0<=i<j<=k<n for which XOR(arr[i..j-1]) equals XOR(arr[j..k]). Each side must contain at least one element.
Constraints: 1 <= arr.length <= 300; 1 <= arr[i] <= 10^8

```
class Solution {
public:
  int countTriplets(vector<int>& arr) {
    unordered_map<int, pair<long long, long long>>
      seen;
    seen[0] = {1, 0};
    int prefix = 0;
    long long answer = 0;
    for (int p = 1; p <=
      static_cast<int>(arr.size()); ++p) {
      prefix ^= arr[p - 1];
      auto& [count, sum] = seen[prefix];
      answer += count * (p - 1) - sum;
      ++count;
      sum += p;
    }
    return static_cast<int>(answer);
  }
};
```

05 LC 268 | Missing Number (Easy)
Problem: Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.
Constraints: n == nums.length; 1 <= n <= 10^4; 0 <= nums[i] <= n; All the numbers of nums are unique.

```
class Solution {
public:
  int missingNumber(vector<int>& nums) {
    int n = nums.size();
    int ans = n;
    for (int i = 0; i < n; ++i) {
      ans ^= (i ^ nums[i]);
    }
    return ans;
  }
};
```

18 LC 9 | Palindrome Number (Easy)
Problem: Given an integer x, return true if x is a palindrome, and false otherwise.
Constraints: -2^31 <= x <= 2^31 - 1

```
class Solution {
public:
  bool isPalindrome(int x) {
    if (x < 0 || (x && x % 10 == 0)) {
      return false;
    }
    int y = 0;
    for (; y < x; x /= 10) {
      y = y * 10 + x % 10;
    }
    return x == y || x == y / 10;
  }
};
```

35 LC 119 | Pascal's Triangle II (Easy)
Problem: Given an integer rowIndex, return the rowIndex^th (0-indexed) row of the Pascal's triangle. In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:
Constraints: 0 <= rowIndex <= 33

```
class Solution {
public:
  vector<int> getRow(int rowIndex) {
    vector<int> f(rowIndex + 1, 1);
    for (int i = 2; i < rowIndex + 1; ++i) {
      for (int j = i - 1; j; --j) {
        f[j] += f[j - 1];
      }
    }
    return f;
  }
};
```

43 LC 2433 | Find The Original Array of Prefix Xor (Medium)
Problem: You are given an integer array pref of size n. Find and return the array arr of size n that satisfies: - pref[i] = arr[0] ^ arr[1] ^ ... ^ arr[i]. Note that ^ denotes the bitwise-xor operation. It can be proven that the answer is unique.
Constraints: 1 <= pref.length <= 10^5; 0 <= pref[i] <= 10^6

```
class Solution {
public:
  vector<int> findArray(vector<int>& pref) {
    int n = pref.size();
    vector<int> ans = {pref[0]};
    for (int i = 1; i < n; ++i) {
      ans.push_back(pref[i - 1] ^ pref[i]);
    }
    return ans;
  }
};
```

06 LC 371 | Sum of Two Integers (Medium)
Problem: Given two integers a and b, return the sum of the two integers without using the operators + and -.
Constraints: -1000 <= a, b <= 1000

```
class Solution {
public:
  int getSum(int a, int b) {
    while (b) {
      unsigned int carry = (unsigned int) (a & b)
        << 1;
      a = a ^ b;
      b = carry;
    }
    return a;
  }
};
```

23 LC 172 | Factorial Trailing Zeroes (Medium)
Problem: Given an integer n, return the number of trailing zeroes in n!. Note that n! = n * (n - 1) * (n - 2) * ... * 3 * 2 * 1.
Constraints: 0 <= n <= 10^4

```
class Solution {
public:
  int trailingZeroes(int n) {
    int ans = 0;
    while (n) {
      n /= 5;
      ans += n;
    }
    return ans;
  }
};
```

38 LC 89 | Gray Code (Medium)
Problem: Return a length-2^n Gray-code sequence beginning with 0. Every value in [0,2^n-1] occurs once, and adjacent values, including the final/first pair, differ in exactly one bit.
Constraints: 1 <= n <= 16

```
class Solution {
public:
  vector<int> grayCode(int n) {
    vector<int> ans;
    for (int i = 0; i < 1 << n; ++i) {
      ans.push_back(i ^ (i >> 1));
    }
    return ans;
  }
};
```

40 LC 1863 | Sum of All Subset XOR Totals (Easy)
Problem: Return the sum of the XORs of every subset of nums. Each index is independently selected, so equal values at different indices count as separate choices. The empty subset contributes 0.
Constraints: 1 <= nums.length <= 12; 1 <= nums[i] <= 20

```
class Solution {
public:
  int subsetXORSum(vector<int>& nums) {
    unsigned bits = 0;
    for (int x : nums) bits |=
      static_cast<unsigned>(x);
    return static_cast<int>(bits << (nums.size() -
      1));
  }
};
```

07 LC 201 | Bitwise AND of Numbers Range (Medium)
Problem: Given two integers left and right that represent the range [left, right], return the bitwise AND of all numbers in this range, inclusive.
Constraints: 0 <= left <= right <= 2^31 - 1

```
class Solution {
public:
  int rangeBitwiseAnd(int left, int right) {
    while (left < right) {
      right &= (right - 1);
    }
    return right;
  }
};
```

08 LC 231 | Power of Two (Easy)
Problem: Given an integer n, return true if it is a power of two. Otherwise, return false. An integer n is a power of two, if there exists an integer x such that n == 2^x.
Constraints: -2^31 <= n <= 2^31 - 1

```
class Solution {
public:
  bool isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
  }
};
```

10 LC 326 | Power of Three (Easy)
Problem: Given an integer n, return true if it is a power of three. Otherwise, return false. An integer n is a power of three, if there exists an integer x such that n == 3^x.
Constraints: -2^31 <= n <= 2^31 - 1

```
class Solution {
public:
  bool isPowerOfThree(int n) {
    return n > 0 && 1162261467 % n == 0;
  }
};
```

36 LC 62 | Unique Paths (Medium)
Problem: A robot starts at the top-left of an m-by-n grid and moves only right or down. Return the number of paths to the bottom-right.
Constraints: 1 <= m, n <= 100

```
class Solution {
public:
  int uniquePaths(int m, int n) {
    int k = min(m - 1, n - 1), total = m + n - 2;
    long long paths = 1;
    for (int i = 1; i <= k; ++i) paths = paths *
      (total - k + i) / i;
    return static_cast<int>(paths);
  }
};
```

37 LC 96 | Unique Binary Search Trees (Medium)
Problem: Given an integer n, return the number of structurally unique BST's (binary search trees) which has exactly n nodes of unique values from 1 to n.
Constraints: 1 <= n <= 19

```
class Solution {
public:
  int numTrees(int n) {
    long long count = 1;
    for (int i = 0; i < n; ++i) count = count * 2
      * (2 * i + 1) / (i + 2);
    return static_cast<int>(count);
  }
};
```

09 LC 342 | Power of Four (Easy)
Problem: Given an integer n, return true if it is a power of four. Otherwise, return false. An integer n is a power of four, if there exists an integer x such that n == 4^x.
Constraints: -2^31 <= n <= 2^31 - 1

```
class Solution {
public:
  bool isPowerOfFour(int n) {
    return n > 0 && (n & (n - 1)) == 0 && (n &
      0xaaaaaaaa) == 0;
  }
};
```

21 LC 204 | Count Primes (Medium)
Problem: Given an integer n, return the number of prime numbers that are strictly less than n.
Constraints: 0 <= n <= 5 * 10^6

```
class Solution {
public:
  int countPrimes(int n) {
    vector<bool> primes(n, true);
    int ans = 0;
    for (int i = 2; i < n; ++i) {
      if (primes[i]) {
        ++ans;
        for (int j = i; j < n; j += i) primes[j] =
          false;
      }
    }
    return ans;
  }
};
```

22 LC 202 | Happy Number (Easy)
Problem: Repeatedly replace positive n with the sum of its squared decimal digits. Return true if this process reaches 1; return false if it enters a different cycle.
Constraints: 1 <= n <= 2^31 - 1

```
class Solution {
public:
  bool isHappy(int n) {
    unordered_set<int> vis;
    while (n != 1 && !vis.count(n)) {
      vis.insert(n);
      int x = 0;
      for (; n; n /= 10) {
        x += (n % 10) * (n % 10);
      }
      n = x;
    }
    return n == 1;
  }
};
```

32 LC 168 | Excel Sheet Column Title (Easy)
Problem: Given an integer columnNumber, return its corresponding column title as it appears in an Excel sheet. For example: A -> 1 B -> 2 C -> 3 ... Z -> 26 AA -> 27 AB -> 28 ...
Constraints: 1 <= columnNumber <= 2^31 - 1

```
class Solution {
public:
  string convertToTitle(int columnNumber) {
    string result;
    while (columnNumber) {
      --columnNumber;
      result += static_cast<char>('A' +
        columnNumber % 26);
      columnNumber /= 26;
    }
    ranges::reverse(result);
    return result;
  }
};
```

12 LC 477 | Total Hamming Distance (Medium)

Problem: The Hamming distance between two integers is the number of positions at which the corresponding bits are different. Given an integer array nums, return the sum of Hamming distances between all the pairs of the integers in nums.

Constraints: 1 <= nums.length <= 10^4; 0 <= nums[i] <= 10^9; The answer for the given input will fit in a 32-bit integer.

```
class Solution {
public:
  int totalHammingDistance(vector<int>& nums) {
    int ans = 0, n = nums.size();
    for (int i = 0; i < 32; ++i) {
      int a = 0;
      for (int x : nums) {
        a += x >> i & 1;
      }
      int b = n - a;
      ans += a * b;
    }
    return ans;
  }
};
```

19 LC 50 | Pow(x, n) (Medium)

Problem: Implement pow(x, n), which calculates x raised to the power n (i.e., x^n).

Constraints: -100.0 < x < 100.0; -2^31 <= n <= 2^31-1; n is an integer.; Either x is not zero or n > 0.; -10^4 <= x^n <= 10^4

```
class Solution {
public:
  double myPow(double x, int n) {
    auto qpow = [](double a, long long n) {
      double ans = 1;
      for (; n; n >>= 1) {
        if (n & 1) {
          ans *= a;
        }
        a *= a;
      }
      return ans;
    };
    return n >= 0 ? qpow(x, n) : 1 / qpow(x,
      -(long long) n);
  }
};
```

20 LC 69 | Sqrt(x) (Easy)

Problem: Given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well. You must not use any built-in exponent function or operator. - For example, do not use pow(x, 0.5) in c++ or x 0.5 in python.

Constraints: 0 <= x <= 2^31 - 1

```
class Solution {
public:
  int mySqrt(int x) {
    int l = 0, r = x;
    while (l < r) {
      int mid = (l + r + 1ll) >> 1;
      if (mid > x / mid) {
        r = mid - 1;
      } else {
        l = mid;
      }
    }
    return l;
  }
};
```

13 LC 137 | Single Number II (Medium)

Problem: Given an integer array nums where every element appears three times except for one, which appears exactly once. Find the single element and return it. You must implement a solution with a linear runtime complexity and use only constant extra space.

Constraints: 1 <= nums.length <= 3 * 10^4; -2^31 <= nums[i] <= 2^31 - 1; Each element in nums appears exactly three times except for one element which appears once.

```
class Solution {
public:
  int singleNumber(vector<int>& nums) {
    int a = 0, b = 0;
    for (int c : nums) {
      int aa = (~a & b & c) | (a & ~b & ~c);
      int bb = ~a & (b ^ c);
      a = aa;
      b = bb;
    }
    return b;
  }
};
```

25 LC 67 | Add Binary (Easy)

Problem: Given two binary strings a and b, return their sum as a binary string.

Constraints: 1 <= a.length, b.length <= 10^4; a and b consist only of '0' or '1' characters.; Each string does not contain leading zeros except for the zero itself.

```
class Solution {
public:
  string addBinary(string a, string b) {
    string ans;
    int i = a.size() - 1, j = b.size() - 1;
    for (int carry = 0; i >= 0 || j >= 0 || carry;
      --i, --j) {
      carry += (i >= 0 ? a[i] - '0' : 0) + (j >= 0
        ? b[j] - '0' : 0);
      ans.push_back((carry % 2) + '0');
      carry /= 2;
    }
    ranges::reverse(ans);
    return ans;
  }
};
```

34 LC 118 | Pascal's Triangle (Easy)

Problem: Given an integer numRows, return the first numRows of Pascal's triangle. In Pascal's triangle, each number is the sum of the two numbers directly above it as shown:

Constraints: 1 <= numRows <= 30

```
class Solution {
public:
  vector<vector<int>> generate(int numRows) {
    vector<vector<int>> f;
    f.push_back(vector<int>(1, 1));
    for (int i = 0; i < numRows - 1; ++i) {
      vector<int> g;
      g.push_back(1);
      for (int j = 1; j < f[i].size(); ++j) {
        g.push_back(f[i][j - 1] + f[i][j]);
      }
      g.push_back(1);
      f.push_back(g);
    }
    return f;
  }
};
```

15 LC 287 | Find the Duplicate Number (Medium)

Problem: Given an array of integers nums containing n + 1 integers where each integer is in the range [1, n] inclusive. There is only one repeated number in nums, return this repeated number. You must solve the problem without modifying the array nums and using only constant extra space.

Constraints: 1 <= n <= 10^5; nums.length == n + 1; 1 <= nums[i] <= n; All the integers in nums appear only once except for precisely one integer which appears two or more times.

```
class Solution {
public:
  int findDuplicate(vector<int>& nums) {
    int slow = 0, fast = 0;
    do { slow = nums[slow]; fast =
      nums[nums[fast]]; } while (slow != fast);
    slow = 0;
    while (slow != fast) { slow = nums[slow]; fast
      = nums[fast]; }
    return slow;
  }
};
```

17 LC 7 | Reverse Integer (Medium)

Problem: Given a signed 32-bit integer x, return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range [-2^31, 2^31 - 1], then return 0. Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Constraints: -2^31 <= x <= 2^31 - 1

```
class Solution {
public:
  int reverse(int x) {
    int ans = 0;
    for (; x; x /= 10) {
      if (ans < INT_MIN / 10 || ans > INT_MAX /
        10) {
        return 0;
      }
      ans = ans * 10 + x % 10;
    }
    return ans;
  }
};
```

24 LC 66 | Plus One (Easy)

Problem: You are given a large integer represented as an integer array digits, where each digits[i] is the ith digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's. Increment the large integer by one and return the resulting array of digits.

Constraints: 1 <= digits.length <= 100; 0 <= digits[i] <= 9; digits does not contain any leading 0's.

```
class Solution {
public:
  vector<int> plusOne(vector<int>& digits) {
    for (int i = digits.size() - 1; i >= 0; --i) {
      ++digits[i];
      digits[i] %= 10;
      if (digits[i] != 0) return digits;
    }
    digits.insert(digits.begin(), 1);
    return digits;
  }
};
```

16 LC 29 | Divide Two Integers (Medium)

Problem: Divide dividend by divisor without multiplication, division, or remainder operators. Truncate toward zero. Clamp a result outside signed 32-bit range to the nearest endpoint.

Constraints: -2^31 <= dividend, divisor <= 2^31 - 1; divisor != 0

```
class Solution {
public:
  int divide(int a, int b) {
    if (b == 1) {
      return a;
    }
    if (a == INT_MIN && b == -1) {
      return INT_MAX;
    }
    bool sign = (a > 0 && b > 0) || (a < 0 && b <
      0);
    a = a > 0 ? -a : a;
    b = b > 0 ? -b : b;
    int ans = 0;
    while (a <= b) {
      int x = b;
      int cnt = 1;
      while (x >= -(1 << 30) && a <= x + x) {
        x += x;
        cnt <<= 1;
      }
      ans += cnt;
      a -= x;
    }
    return sign ? ans : -ans;
  }
};
```

50 LC 593 | Valid Square (Medium)

Problem: Given the coordinates of four points in 2D space p1, p2, p3 and p4, return true if the four points construct a square. The coordinate of a point p_i is represented as [x_i, y_i]. The input is not given in any order. A valid square has four equal sides with positive length and four equal angles (90-degree angles).

Constraints: p1.length == p2.length == p3.length == p4.length == 2; -10^4 <= x_i, y_i <= 10^4

```
class Solution {
public:
  bool validSquare(vector<int>& p1, vector<int>&
    p2, vector<int>& p3, vector<int>& p4) {
    array<const vector<int>*, 4> points{&p1, &p2,
      &p3, &p4};
    array<long long, 6> distances{};
    int next = 0;
    for (int i = 0; i < 4; ++i) for (int j = i +
      1; j < 4; ++j) {
      long long dx = (*points[i])[0] -
        (*points[j])[0];
      long long dy = (*points[i])[1] -
        (*points[j])[1];
      distances[next++] = dx * dx + dy * dy;
    }
    ranges::sort(distances);
    return distances[0] > 0 && distances[0] ==
      distances[1] && distances[1] == distances[2]
      &&
      distances[2] == distances[3] && distances[4]
        == distances[5] && distances[4] == 2 *
        distances[0];
  }
};
```

26 LC 43 | Multiply Strings (Medium)

Problem: Given two non-negative integers num1 and num2 represented as strings, return the product of num1 and num2, also represented as a string. Note: You must not use any built-in BigInteger library or convert the inputs to integer directly.
Constraints: 1 <= num1.length, num2.length <= 200; num1 and num2 consist of digits only.; Both num1 and num2 do not contain any leading zero, except the number 0 itself.

```
class Solution {
public:
  string multiply(string num1, string num2) {
    if (num1 == "0" || num2 == "0") {
      return "0";
    }
    int m = num1.size(), n = num2.size();
    vector<int> arr(m + n);
    for (int i = m - 1; i >= 0; --i) {
      int a = num1[i] - '0';
      for (int j = n - 1; j >= 0; --j) {
        int b = num2[j] - '0';
        arr[i + j + 1] += a * b;
      }
    }
    for (int i = arr.size() - 1; i; --i) {
      arr[i - 1] += arr[i] / 10;
      arr[i] %= 10;
    }
    int i = arr[0] ? 0 : 1;
    string ans;
    for (; i < arr.size(); ++i) {
      ans += '0' + arr[i];
    }
    return ans;
  }
};
```

47 LC 2013 | Detect Squares (Medium)

Problem: Implement DetectSquares.add(point) and count(point). Count choices of three previously added points that, with the queried point, form a positive-area axis-aligned square. Repeated added coordinates count as distinct choices; a query does not add its point.
Constraints: point.length == 2; 0 <= x, y <= 1000; At most 3000 calls in total will be made to add and count.

```
class DetectSquares {
public:
  DetectSquares() {
  }
  void add(vector<int> point) {
    int x = point[0], y = point[1];
    ++cnt[x][y];
  }
  int count(vector<int> point) {
    int x1 = point[0], y1 = point[1];
    if (!cnt.count(x1)) {
      return 0;
    }
    int ans = 0;
    for (auto& [x2, cnt2] : cnt) {
      if (x2 != x1) {
        int d = x2 - x1;
        auto& cnt1 = cnt[x1];
        ans += cnt2[y1] * cnt1[y1 + d] * cnt2[y1 +
          d];
        ans += cnt2[y1] * cnt1[y1 - d] * cnt2[y1 -
          d];
      }
    }
    return ans;
  }
private:
  unordered_map<int, unordered_map<int, int>> cnt;
};
```

28 LC 989 | Add to Array-Form of Integer (Easy)

Problem: The array-form of an integer num is an array representing its digits in left to right order. - For example, for num = 1321, the array form is [1,3,2,1]. Given num, the array-form of an integer, and an integer k, return the array-form of the integer num + k.
Constraints: 1 <= num.length <= 10^4; 0 <= num[i] <= 9; num does not contain any leading zeros except for the zero itself.; 1 <= k <= 10^4

```
class Solution {
public:
  vector<int> addToArrayForm(vector<int>& num, int
    k) {
    vector<int> ans;
    for (int i = num.size() - 1; i >= 0 || k > 0;
      --i) {
      k += (i >= 0 ? num[i] : 0);
      ans.push_back(k % 10);
      k /= 10;
    }
    ranges::reverse(ans);
    return ans;
  }
};
```

31 LC 12 | Integer to Roman (Medium)

Problem: Convert num to a canonical Roman numeral using I=1, V=5, X=10, L=50, C=100, D=500, M=1000 and subtractive pairs IV, IX, XL, XC, CD, CM.
Constraints: 1 <= num <= 3999

```
class Solution {
public:
  string intToRoman(int num) {
    vector<string> cs = {"M", "CM", "D", "CD",
      "C", "XC", "L", "XL", "X", "IX", "V", "IV",
      "I"};
    vector<int> vs = {1000, 900, 500, 400, 100,
      90, 50, 40, 10, 9, 5, 4, 1};
    string ans;
    for (int i = 0; i < cs.size(); ++i) {
      while (num >= vs[i]) {
        num -= vs[i];
        ans += cs[i];
      }
    }
    return ans;
  }
};
```

45 LC 1310 | XOR Queries of a Subarray (Medium)

Problem: You are given an array arr of positive integers. You are also given the array queries where queries[i] = [left_i, right_i]. For each query i compute the XOR of elements from left_i to right_i (that is, arr[left_i] XOR arr[left_i + 1] XOR ... XOR arr[right_i]). Return an array answer where answer[i] is the answer to the ith query.
Constraints: 1 <= arr.length, queries.length <= 3 * 10^4; 1 <= arr[i] <= 10^9; queries[i].length == 2; 0 <= left_i <= right_i < arr.length

```
class Solution {
public:
  vector<int> xorQueries(vector<int>& arr,
    vector<vector<int>>& queries) {
    int n = arr.size();
    vector<int> s = vector<int>(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
      s[i] = s[i - 1] ^ arr[i - 1];
    }
    vector<int> ans;
    for (auto& q : queries) {
      int l = q[0], r = q[1];
      ans.push_back(s[r + 1] ^ s[l]);
    }
    return ans;
  }
};
```

29 LC 8 | String to Integer (atoi) (Medium)

Problem: Convert s manually to a signed 32-bit integer: skip leading spaces, read an optional sign, then consecutive decimal digits until the first non-digit or the end. Return 0 if no digits are read; clamp overflow to [-2^31, 2^31-1]. Do not use a built-in string-to-number conversion.

Constraints: 0 <= s.length <= 200; s consists of English letters (lower-case and upper-case), digits (0 - 9), ' ', '+', '-', and '.'.

```
class Solution {
public:
  int myAtoi(string s) {
    int i = 0, n = s.size();
    while (i < n && s[i] == ' ')
      ++i;
    int sign = 1;
    if (i < n && (s[i] == '-' || s[i] == '+')) {
      sign = s[i] == '-' ? -1 : 1;
      ++i;
    }
    int res = 0;
    while (i < n && isdigit(s[i])) {
      int digit = s[i] - '0';
      if (res > INT_MAX / 10 || (res == INT_MAX /
        10 && digit > INT_MAX % 10)) {
        return sign == 1 ? INT_MAX : INT_MIN;
      }
      res = res * 10 + digit;
      ++i;
    }
    return res * sign;
  }
};
```

30 LC 13 | Roman to Integer (Easy)

Problem: Convert a valid Roman numeral s to an integer. Symbols I,V,X,L,C,D,M represent 1,5,10,50,100,500,1000; subtractive pairs IV,IX,XL,XC,CD,CM represent 4,9,40,90,400,900.

Constraints: 1 <= s.length <= 15; s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').; It is guaranteed that s is a valid roman numeral in the range [1, 3999].

```
class Solution {
public:
  int romanToInt(string s) {
    unordered_map<char, int> nums{
      {'I', 1},
      {'V', 5},
      {'X', 10},
      {'L', 50},
      {'C', 100},
      {'D', 500},
      {'M', 1000},
    };
    int ans = nums[s.back()];
    for (int i = 0; i < s.size() - 1; ++i) {
      int sign = nums[s[i]] < nums[s[i + 1]] ? -1
        : 1;
      ans += sign * nums[s[i]];
    }
    return ans;
  }
};
```

39 LC 78 | Subsets (Medium)

Problem: Given an integer array nums of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets. Return the solution in any order.

Constraints: 1 <= nums.length <= 10; -10 <= nums[i] <= 10; All the numbers of nums are unique.

```
class Solution {
public:
  vector<vector<int>> subsets(vector<int>& nums) {
    vector<vector<int>> ans;
    vector<int> t;
    function<void(int)> dfs = [&](int i) -> void {
      if (i == nums.size()) {
        ans.push_back(t);
        return;
      }
      dfs(i + 1);
      t.push_back(nums[i]);
      dfs(i + 1);
      t.pop_back();
    };
    dfs(0);
    return ans;
  }
};
```

46 LC 421 l Maximum XOR of Two Numbers in an Arrav (Medium)

Problem: Given an integer array nums, return the maximum result of nums[i] XOR nums[j], where 0 <= i <= j < n.

Constraints: 1 <= nums.length <= 2 * 10^5; 0 <= nums[i] <= 2^31 - 1

```
class Trie {
public:
  Trie* children[2];
  Trie()
    : children{nullptr, nullptr} {}
  void insert(int x) {
    Trie* node = this;
    for (int i = 30; ~i; --i) {
      int v = x >> i & 1;
      if (!node->children[v]) {
        node->children[v] = new Trie();
      }
      node = node->children[v];
    }
  }
  int search(int x) {
    Trie* node = this;
    int ans = 0;
    for (int i = 30; ~i; --i) {
      int v = x >> i & 1;
      if (node->children[v ^ 1]) {
        ans |= 1 << i;
        node = node->children[v ^ 1];
      } else {
        node = node->children[v];
      }
    }
    return ans;
  }
};
class Solution {
public:
  int findMaximumXOR(vector<int>& nums) {
    Trie* trie = new Trie();
    int ans = 0;
    for (int x : nums) {
      trie->insert(x);
      ans = max(ans, trie->search(x));
    }
    return ans;
  }
};
```